

Praktikum 10 – Machine Learning K-Nearest Neighbors (KNN)

Prepared By:

Dr. Sirojul Munir S.Si,M.Kom

Diah Ayu Puspasari

Elyas Randi Renaldi

Tujuan :

1. Menjelaskan konsep dasar algoritma K-Nearest Neighbors (KNN)
2. Mengimplementasikannya untuk kasus supervised learning dalam memprediksi kelulusan mahasiswa berdasarkan data kelulusan_train.xls dan kelulusan_test.xls, mulai dari tahap preprocessing, pemisahan data training dan testing, pembangunan model KNN, hingga evaluasi performa model menggunakan Confusion Matrix, Classification Report, dan akurasi model.

Dateline : 1 Pekan

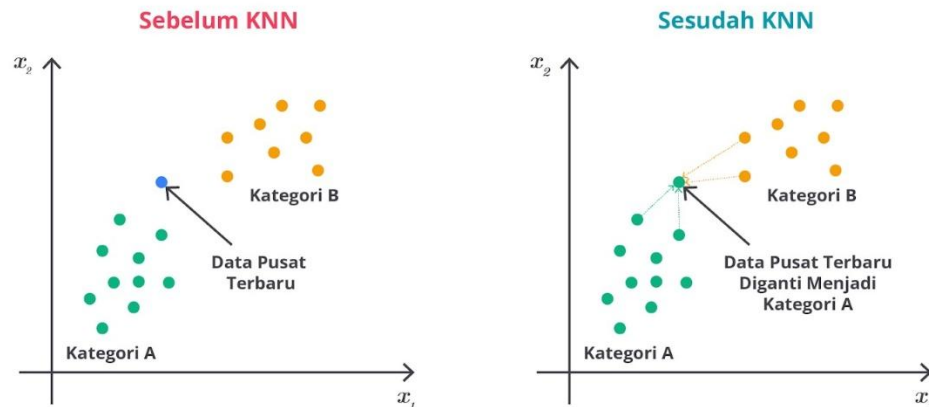
Gitlab/Github :

Branch Repository : [PRODI ROMBEL]_[NAMASINGKAT]_[NIM] (contoh: ti01_budi_0110112001)

Aturan Pengerjaan:

1. Gunakan text editor yang nyaman bagi anda
2. Diperkenankan mengerjakan langsung bagi yang sudah memahami dan menguasai materi
3. Dilarang melakukan tindakan plagiarisme (asisten lab akan mengecek hasil pekerjaan)
 - a. 1x nilai praktikum terkait bernilai 0
 - b. 2x nilai matakuliah pemrograman web E
 - c. 3x mahasiswa akan di sidang komite etik kampus

Pendahuluan



Algoritma K-Nearest Neighbors (KNN) adalah metode supervised learning yang digunakan untuk mengatasi masalah klasifikasi dan regresi. Evelyn Fix dan Joseph Hodges mengembangkan algoritma ini pada tahun 1951 yang kemudian diperluas oleh Thomas Cover. KNN merupakan salah satu algoritma klasifikasi yang paling sederhana dan intuitif dalam machine learning.

Algoritma ini digunakan untuk mengklasifikasikan data baru berdasarkan kedekatannya dengan data yang sudah diberi label dalam dataset pelatihan. KNN sering digunakan karena kemudahannya dalam pemahaman dan implementasi meskipun pada praktiknya, ia dapat menjadi sangat efektif untuk berbagai masalah klasifikasi.

Prinsip Kerja:

1. Menentukan jumlah tetangga terdekat (**K**) yang akan digunakan.
2. Menghitung jarak antara data baru dengan seluruh data pada dataset training (biasanya menggunakan **Euclidean Distance**).
3. Memilih **K data terdekat** berdasarkan jarak terkecil.
4. Untuk kasus klasifikasi, label mayoritas dari K data terdekat tersebut akan menjadi **prediksi label** untuk data baru.

Perbedaan dengan K-Means:

Berbeda dengan K-Means yang bersifat unsupervised learning (tidak menggunakan label saat training), KNN merupakan algoritma supervised learning. Artinya, KNN membutuhkan data dengan label (target) untuk belajar dan membuat prediksi terhadap data baru.

Langkah Awal:

Sebelum memulai praktikum ini, pastikan Anda telah menyiapkan struktur folder di Google Drive / Jupyter seperti pada praktikum sebelumnya. Untuk praktikum ke-10, buat sub-folder baru dengan nama praktikum10/ di dalam direktori utama praktikum_ml/.

Struktur folder yang digunakan adalah sebagai berikut:

```
praktikum_ml/  
├─ praktikum01/  
├─ praktikum02/  
├─ praktikum03/  
│   └─ data/  
│   └─ notebooks/  
│   └─ model/  
│   └─ reports/
```

Catatan:

1. Jika telah memiliki folder praktikum_ml, cukup tambahkan folder praktikum10.
2. Semua file notebook pada praktikum ini disimpan di dalam folder notebooks/ dengan nama praktikum10.ipynb.
3. Dataset ditempatkan di folder data/.
4. Model hasil training disimpan di folder model/ (format .pkl atau .joblib).
5. Visualisasi atau laporan hasil analisis disimpan di folder reports/.Jalankan file installer yang sudah diunduh.

Setelah itu, lakukan Langkah-langkah seperti biasa untuk Loading dataset. Mulai dari menghubungkannya dengan google drive sampai membaca dataset.

Praktikum K-Nearest Neighbors (KNN)

1. Import Library

Tahapan pertama pada praktikum ini adalah mengimpor library atau pustaka yang akan digunakan dalam proses analisis dan pembuatan model K-Nearest Neighbors (KNN).

Setiap library memiliki fungsi spesifik yang membantu dalam pembacaan data, preprocessing, visualisasi, pelatihan model, dan evaluasi performa.

Library / Modul	Fungsi / Kegunaan
pandas (pd)	Membaca, mengolah, dan menganalisis data dalam bentuk DataFrame.
numpy (np)	Melakukan operasi numerik dan manipulasi array.
matplotlib.pyplot (plt)	Membuat grafik dan visualisasi data seperti scatter plot, bar chart, dll.
seaborn (sns)	Membuat visualisasi data yang lebih menarik dan mudah dibaca.
train_test_split	Membagi dataset menjadi data latih (training) dan data uji (testing).

GridSearchCV	Melakukan pencarian parameter terbaik (hyperparameter tuning) untuk model.
cross_val_score	Melakukan validasi silang untuk mengukur performa model.
StandardScaler	Menstandarkan skala fitur agar memiliki mean 0 dan standar deviasi 1.
KNeighborsClassifier	Mengimplementasikan algoritma K-Nearest Neighbors untuk klasifikasi.
classification_report	Menampilkan metrik evaluasi seperti presisi, recall, dan f1-score.
confusion_matrix	Menampilkan matriks perbandingan hasil prediksi dan label sebenarnya.
PCA (Principal Component Analysis)	Melakukan reduksi dimensi untuk visualisasi data dalam 2D atau 3D.
SMOTE (Synthetic Minority Over-sampling Technique)	Menangani ketidakseimbangan kelas dengan membuat data sintetis untuk kelas minoritas.

2. Loading dataset

Pada tahap ini, dilakukan proses menghubungkan Google Colab dengan Google Drive agar file dataset yang tersimpan di Drive dapat diakses langsung dari lingkungan kerja Colab.

```
# menghubungkan colab dengan google drive
from google.colab import drive
drive.mount('/content/gdrive')

Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive.mount("/content/gdrive", force_remount=True).

# memanggil data set lewat gdrive
path = "/content/gdrive/MyDrive/praktikum_ml/praktikum10"
```

Kode di atas berfungsi untuk memasang (mount) Google Drive ke direktori /content/gdrive.

Ketika dijalankan, Colab akan meminta izin akses ke akun Google agar bisa membaca file yang disimpan di dalam Drive.

- Menampilkan data awal

Setelah folder path ditentukan, tahap selanjutnya adalah membaca file dataset menggunakan library pandas dan menampilkannya untuk memastikan data berhasil dimuat.

Selanjutnya, kedua dataset ditampilkan dengan fungsi `.head()` untuk melihat **5 baris pertama** dari masing-masing data.

`df_train.head()`

	NAMA	JENIS KELAMIN	STATUS MAHASISWA	UMUR
0	UNAMA	LAKI - LAKI	MAHASISWA	24
1	LEYLA TRIYANA PRATIWI	PEREMPUAN	MAHASISWA	26
2	VERIS SOFIYAN PRAYOGA	LAKI - LAKI	MAHASISWA	29
3	ADITYA AKBAR NUGRAHA	LAKI - LAKI	MAHASISWA	27
4	ERNA EKA RIYANTI	PEREMPUAN	MAHASISWA	25

Next steps: [Generate code with df_train](#) [New interactive sheet](#)

`df_test.head()`

	NAMA	JENIS KELAMIN	STATUS MAHASISWA	UMUR
0	UNAMA	LAKI - LAKI	MAHASISWA	24
1	LEYLA TRIYANA PRATIWI	PEREMPUAN	MAHASISWA	26
2	VERIS SOFIYAN PRAYOGA	LAKI - LAKI	MAHASISWA	29
3	ADITYA AKBAR NUGRAHA	LAKI - LAKI	MAHASISWA	27
4	ERNA EKA RIYANTI	PEREMPUAN	MAHASISWA	25

- Mengecek Struktur Dataset

Setelah dataset berhasil dimuat, langkah selanjutnya adalah memeriksa informasi umum dari data, dengan `.info()`

`df_train.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 145 entries, 0 to 144
Data columns (total 15 columns):
#   Column              Non-Null Count  Dtype
---
```

`df_test.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 145 entries, 0 to 144
Data columns (total 15 columns):
```

Hasil di atas menunjukkan bahwa baik **df_train** maupun **df_test** memiliki **145 baris** dan **15 kolom**, serta struktur data yang serupa.

3. Data Cleaning

- Pemeriksaan Nilai Unik pada Kolom Kategorikal
Pada tahap pembersihan data (data cleaning), salah satu langkah penting adalah memeriksa nilai unik dari setiap kolom kategorikal untuk memastikan tidak ada nilai yang aneh, kosong, atau tidak konsisten penulisannya.

Fungsi `.unique()` digunakan untuk menampilkan **daftar nilai unik** yang terdapat pada suatu kolom.

```
print(df_train['JENIS KELAMIN'].unique())
print(df_train['STATUS MAHASISWA'].unique())
print(df_train['STATUS NIKAH'].unique())
print(df_train['STATUS KELULUSAN'].unique())

... ['LAKI - LAKI' 'PEREMPUAN']
    ['MAHASISWA' 'BEKERJA']
    ['BELUM MENIKAH']
    ['TEPAT' 'TERLAMBAT']
```

- Menghapus Kolom yang Tidak Digunakan

Setelah diperiksa sebelumnya, kolom STATUS NIKAH hanya memiliki satu nilai unik, yaitu BELUM MENIKAH. Kolom seperti ini tidak memberikan informasi yang berguna untuk proses pembelajaran model karena tidak memiliki variasi data (hanya satu kelas saja).

```
df_train.drop(columns=['STATUS NIKAH'], inplace=True)
df_test.drop(columns=['STATUS NIKAH'], inplace=True)
```

Oleh karena itu, kolom tersebut dihapus dari kedua dataset, yaitu data training dan data testing, menggunakan perintah berikut:

- Mengubah Nilai Kategorikal menjadi Numerik

```
replacements = {
    'JENIS KELAMIN': {'LAKI - LAKI': 1, 'PEREMPUAN': 0},
    'STATUS MAHASISWA': {'MAHASISWA': 0, 'BEKERJA': 1},
    'STATUS KELULUSAN': {'TERLAMBAT': 1, 'TEPAT': 0}
}
df_train = df_train.replace(replacements, inplace=False)
df_train.head()
```

```
... /tmp/ipython-input-1749631542.py:6: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the
df_train = df_train.replace(replacements, inplace=False)
```

	NAMA	JENIS KELAMIN	STATUS MAHASISWA	UMUR	IPS 1	IPS 2	IPS 3	IPS 4	IPS 5	IPS 6	IPS 7	IPS 8	IPK	STATUS KELULUSAN
0	UNAMA	1	0	24	3.17	2.70	3.23	2.41	3.00	2.47	1.75	0.00	2.75	0
1	LEYLA TRIYANA PRATIWI	0	0	26	3.60	3.50	3.42	2.85	3.31	2.95	2.18	NaN	3.39	0
2	VERIS SOFIYAN PRAYOGA	1	0	29	2.67	2.66	2.93	3.14	2.92	2.64	2.88	0.50	2.81	0
3	ADITYA AKBAR NUGRAHA	1	0	27	2.48	2.86	2.09	2.55	2.55	2.43	2.55	2.17	2.82	0
4	ERNA EKA RIYANTI	0	0	25	3.19	3.08	3.31	2.83	3.36	2.73	3.06	0.00	3.09	0

Karena algoritma K-Nearest Neighbors (KNN) hanya dapat memproses data numerik, maka kolom dengan nilai kategori (teks) perlu diubah menjadi angka. Proses ini disebut encoding, dan di sini digunakan metode manual dengan fungsi `.replace()` dari pandas.

- Melakukan Encoding pada Dataset Uji

```
df_test = df_test.replace(replacements, inplace=False)
df_test.head()
```

```
*** /tmp/ipython-input-1812349956.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the current behavior, use `df_test = df_test.replace(replacements, inplace=False)`
```

	NAMA	JENIS KELAMIN	STATUS MAHASISWA	UMUR	IPS 1	IPS 2	IPS 3	IPS 4	IPS 5	IPS 6	IPS 7	IPS 8	IPK	STATUS KELULUSAN
0	UNAMA	1	0	24	3.17	2.70	3.23	2.41	3.00	2.47	1.75	0.00	2.75	0
1	LEYLA TRIYANA PRATIWI	0	0	26	3.60	3.50	3.42	2.85	3.31	2.95	2.18	NaN	3.39	0
2	VERIS SOFIYAN PRAYOGA	1	0	29	2.67	2.66	2.93	3.14	2.92	2.64	2.88	0.50	2.81	0
3	ADITYA AKBAR NUGRAHA	1	0	27	2.48	2.86	2.09	2.55	2.55	2.43	2.55	2.17	2.82	0
4	ERNA EKA RIYANTI	0	0	25	3.19	3.08	3.31	2.83	3.36	2.73	3.06	0.00	3.09	0

Setelah melakukan penggantian nilai kategorikal menjadi numerik pada data training, langkah yang sama juga perlu dilakukan pada **data testing** agar model dapat memprosesnya dengan benar.

- Cek Missing Value

Langkah ini bertujuan untuk memastikan apakah terdapat nilai yang kosong (missing) dalam dataset yang bisa memengaruhi proses pelatihan model.

```
df_train.isnull().sum()
```

```
***
```

NAMA	0
JENIS KELAMIN	0
STATUS MAHASISWA	0
UMUR	0
IPS 1	0
IPS 2	0
IPS 3	0
IPS 4	0
IPS 5	0
IPS 6	0
IPS 7	0
IPS 8	4
IPK	3
STATUS KELULUSAN	0

dtype: int64

```
df_test.isnull().sum()
```

```
***
```

NAMA	0
JENIS KELAMIN	0
STATUS MAHASISWA	0
UMUR	0
IPS 1	0
IPS 2	0
IPS 3	0
IPS 4	0
IPS 5	0
IPS 6	0
IPS 7	0
IPS 8	4
IPK	3
STATUS KELULUSAN	0

dtype: int64

- Menangani missing value

```
df_train = df_train.dropna(subset=['IPS 8'])
df_train = df_train.dropna(subset=['IPK '])
```

```
df_test = df_test.dropna(subset=['IPS 8'])
df_test = df_test.dropna(subset=['IPK '])
```

Setelah diketahui bahwa terdapat nilai kosong pada kolom **IPS 8** dan **IPK**, langkah selanjutnya adalah **menghapus baris yang mengandung nilai kosong tersebut**. Hal ini dilakukan menggunakan fungsi `.dropna()` dari pandas.

- Mengecek Kembali Missing Value

Setelah dilakukan penghapusan baris yang memiliki nilai kosong pada kolom IPS 8 dan IPK, langkah berikutnya adalah memastikan bahwa seluruh missing value sudah benar-benar hilang.

<pre>df_test.isnull().sum()</pre> ... 0 NAMA 0 JENIS KELAMIN 0 STATUS MAHASISWA 0 UMUR 0 IPS 1 0 IPS 2 0 IPS 3 0 IPS 4 0 IPS 5 0 IPS 6 0 IPS 7 0 IPS 8 4 IPK 3 STATUS KELULUSAN 0 dtype: int64	<pre>df_train.isnull().sum()</pre> ... 0 NAMA 0 JENIS KELAMIN 0 STATUS MAHASISWA 0 UMUR 0 IPS 1 0 IPS 2 0 IPS 3 0 IPS 4 0 IPS 5 0 IPS 6 0 IPS 7 0 IPS 8 0 IPK 0 STATUS KELULUSAN 0 dtype: int64
--	---

- Menghapus Kolom yang Tidak Digunakan dalam Analisis
Karena kolom NAMA hanya berisi teks identitas mahasiswa dan tidak memiliki hubungan dengan prediksi kelulusan, maka kolom ini tidak diperlukan untuk proses analisis maupun korelasi antar variabel.

```
# Kolom yang ingin digunakan untuk korelasi
df_train = df_train.drop(columns=['NAMA'])
df_train.head(1)
```

	JENIS KELAMIN	STATUS MAHASISWA	UMUR	IPS 1	IPS 2	IPS 3	IPS 4	IPS 5	IPS 6	IPS 7	IPS 8	IPK	STATUS KELULUSAN
0	1	0	24	3.17	2.7	3.23	2.41	3.0	2.47	1.75	0.0	2.75	0

Next steps: [Generate code with df_train](#) [New interactive sheet](#)

```
df_test = df_test.drop(columns=['NAMA'])
df_test.head(1)
```

	JENIS KELAMIN	STATUS MAHASISWA	UMUR	IPS 1	IPS 2	IPS 3	IPS 4	IPS 5	IPS 6	IPS 7	IPS 8	IPK	STATUS KELULUSAN
0	1	0	24	3.17	2.7	3.23	2.41	3.0	2.47	1.75	0.0	2.75	0

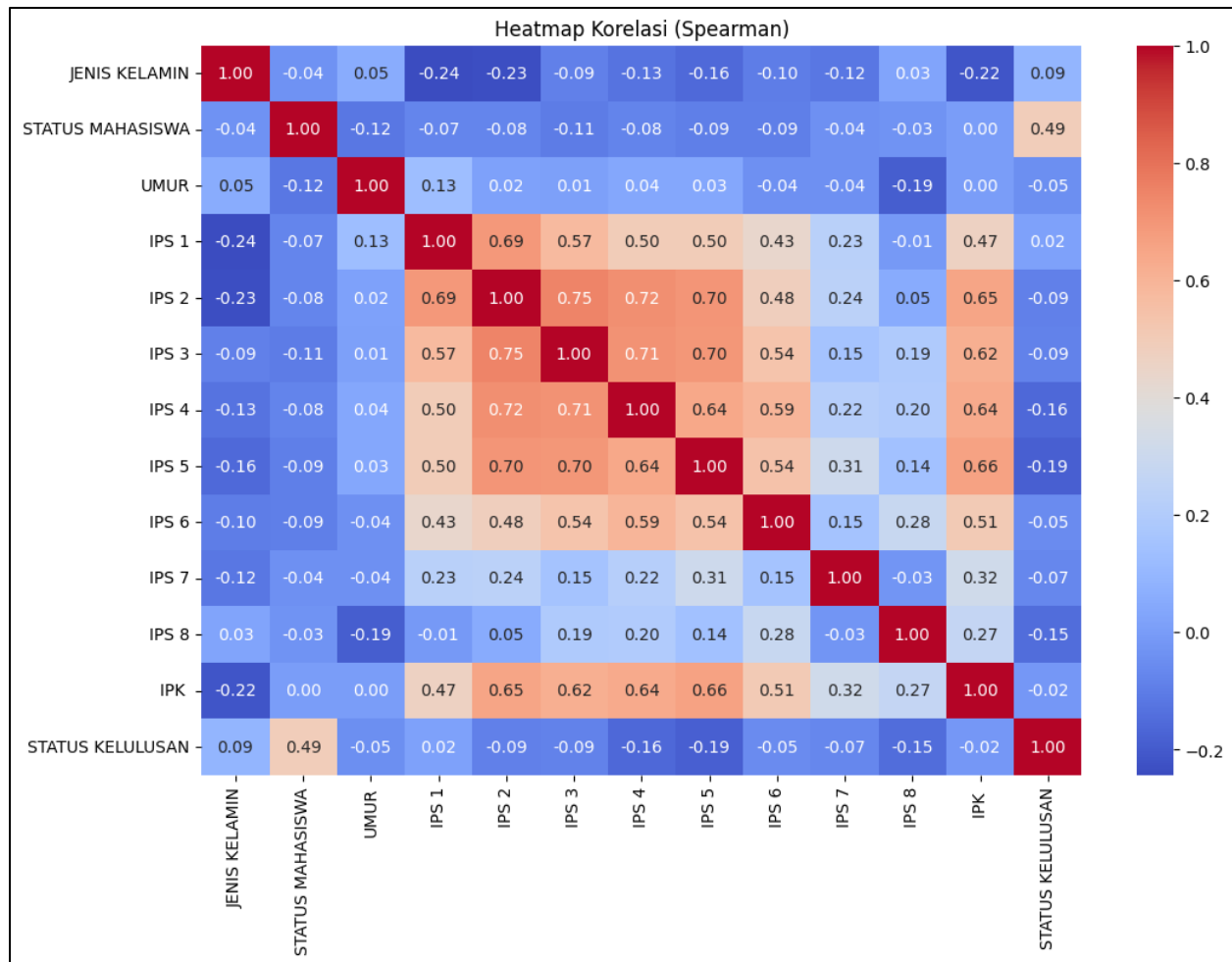
Next steps: [Generate code with df_test](#) [New interactive sheet](#)

4. Analisis Korelasi Fitur

Sebelum melakukan pelatihan model, penting untuk mengetahui apakah ada hubungan (korelasi) antar variabel numerik yang signifikan. Hal ini membantu memahami fitur mana yang paling berpengaruh terhadap Status Kelulusan.

```
# Menghitung matriks korelasi menggunakan Spearman
correlation_matrix = df_train.corr(method='spearman')

# Membuat heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap="coolwarm", cbar=True)
plt.title("Heatmap Korelasi (Spearman)")
plt.show()
```



Hasil Analisis dari Heatmap

- Korelasi antar nilai **IPS 1–IPS 8** cukup tinggi (0.5–0.75), artinya nilai indeks prestasi tiap semester cenderung konsisten antar mahasiswa.
- IPK** memiliki korelasi positif sedang terhadap nilai-nilai IPS (antara 0.47–0.66), yang logis karena IPK merupakan hasil rata-rata dari nilai-nilai IPS tersebut.
- STATUS MAHASISWA** memiliki korelasi **0.49** terhadap **STATUS KELULUSAN**, menunjukkan bahwa status mahasiswa (misalnya bekerja atau tidak) punya pengaruh cukup kuat terhadap ketepatan waktu kelulusan.
- Variabel lain seperti **JENIS KELAMIN** dan **UMUR** memiliki korelasi rendah dengan status kelulusan, artinya keduanya tidak terlalu berpengaruh dalam model prediksi.

5. Visualisasi Distribusi Status Kelulusan

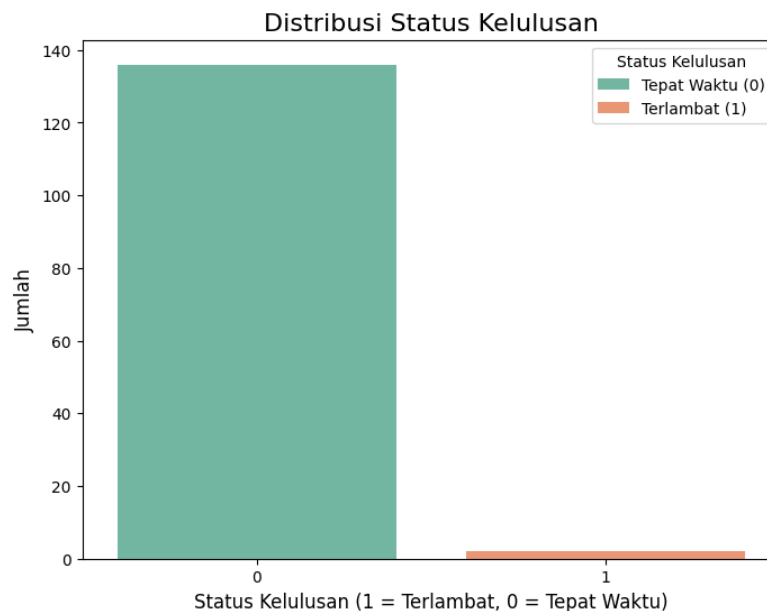
Langkah ini dilakukan untuk melihat sebaran jumlah mahasiswa yang lulus tepat waktu dan yang terlambat. Visualisasi ini membantu memahami apakah data target (label) seimbang atau tidak — hal penting sebelum model KNN dilatih.

```
# Membuat diagram batang untuk status kelulusan
plt.figure(figsize=(8, 6))
sns.countplot(x='STATUS KELULUSAN', data=df_train, palette='Set2')

# Menambahkan judul dan label
plt.title('Distribusi Status Kelulusan', fontsize=16)
plt.xlabel('Status Kelulusan (1 = Terlambat, 0 = Tepat Waktu)', fontsize=12)
plt.ylabel('Jumlah', fontsize=12)

# Menambahkan legend (adjusted labels for correctness)
plt.legend(title='Status Kelulusan', labels=['Tepat Waktu (0)', 'Terlambat (1)'], loc='upper right')

# Menampilkan plot
plt.show()
```



Grafik batang tersebut menunjukkan jumlah mahasiswa yang lulus tepat waktu (0) dan mahasiswa yang terlambat lulus (1).

TAHAP MODELING

6. Pemisahan Data dan Normalisasi

```
# data yang dilatih
x_train = df_train.drop(columns=['STATUS KELULUSAN'])
y_train = df_train['STATUS KELULUSAN']

# data validasinya
x_test = df_test.drop(columns=['STATUS KELULUSAN'])
y_test = df_test['STATUS KELULUSAN']

# Normalisasi features after splitting
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

Sebelum melatih model KNN, dataset perlu dibagi menjadi dua bagian:

1. **Fitur (X)** → variabel input yang digunakan untuk prediksi.
2. **Label (y)** → target atau hasil yang ingin diprediksi (dalam hal ini, STATUS KELULUSAN).

Algoritma **KNN** sangat bergantung pada **jarak antar data**, sehingga setiap fitur harus memiliki skala yang sebanding. Untuk itu dilakukan **normalisasi (standarisasi)** menggunakan StandardScaler.

7. Menangani Ketidakseimbangan Kelas dengan SMOTE

Dari hasil visualisasi sebelumnya, diketahui bahwa dataset sangat tidak seimbang — hanya sedikit data untuk kelas Terlambat (1) dibanding Tepat Waktu (0). Hal ini dapat membuat model KNN bias terhadap kelas mayoritas, sehingga perlu diseimbangkan menggunakan SMOTE.

```
print("Before SMOTE:")
print(y_train.value_counts())

sm = SMOTE(random_state=42, k_neighbors=1) # Set k_neighbors to 1
x_train_smote, y_train_smote = sm.fit_resample(x_train_scaled, y_train)

print("After SMOTE:")
print(y_train_smote.value_counts())

... Before SMOTE:
STATUS KELULUSAN
0    136
1     2
Name: count, dtype: int64
After SMOTE:
STATUS KELULUSAN
0    136
1   136
Name: count, dtype: int64
```

Penjelasan dari kode:

- **SMOTE()** → teknik oversampling yang membuat **data sintetis baru untuk kelas minoritas** dengan cara menginterpolasi data yang sudah ada.
- **fit_resample()** → menyeimbangkan data training sehingga jumlah data tiap kelas sama.

- **random_state=42** → memastikan hasil replikasi tetap sama setiap kali dijalankan.
- **k_neighbors=1** → menentukan jumlah tetangga terdekat yang digunakan untuk membuat sampel baru (diset kecil karena data minoritasnya sangat sedikit).

8. Menentukan Parameter Terbaik (Hyperparameter Tuning)

Agar model KNN memberikan hasil prediksi terbaik, perlu dicari kombinasi parameter yang optimal seperti: jumlah tetangga (**n_neighbors**), jenis pembobotan (**weights**), dan metode perhitungan jarak (**metric**).

```
param_grid = {  
    'n_neighbors': list(range(1, 31)),  
    'weights': ['uniform', 'distance'],  
    'metric': ['euclidean', 'manhattan']  
}  
  
grid = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5, n_jobs=-1)  
grid.fit(x_train_smote, y_train_smote)  
  
print("Best Params:", grid.best_params_)  
print("Best Score (CV):", grid.best_score_)  
  
... Best Params: {'metric': 'euclidean', 'n_neighbors': 1, 'weights': 'uniform'}  
Best Score (CV): 0.9925925925925926
```

Penjelasan:

- **param_grid** → berisi daftar kombinasi parameter yang akan diuji.
 - **n_neighbors**: mencoba nilai K dari 1 sampai 30.
 - **weights**:
 - 'uniform' → semua tetangga punya bobot yang sama.
 - 'distance' → tetangga yang lebih dekat punya bobot lebih besar.
 - **metric**: jenis jarak yang digunakan (Euclidean atau Manhattan).
- **GridSearchCV** → menguji semua kombinasi parameter dengan validasi silang (cv=5) untuk menemukan hasil terbaik.
- **n_jobs=-1** → memanfaatkan semua core CPU agar proses lebih cepat.
- **grid.best_params_** → menampilkan kombinasi parameter terbaik.
- **grid.best_score_** → menampilkan skor rata-rata terbaik dari cross-validation.

Output dari kode tersebut memiliki arti;

Model KNN terbaik menggunakan:

- Jarak Euclidean
- K = 1 tetangga terdekat
- Pembobotan uniform (semua tetangga berbobot sama)

Akurasi rata-rata dari hasil cross-validation sangat tinggi, yaitu sekitar 99.29%, yang menunjukkan model bekerja dengan sangat baik pada data training hasil SMOTE.

9. Melatih Model KNN Terbaik dan Melakukan Prediks

Setelah parameter terbaik ditemukan dengan GridSearchCV, model KNN terbaik tersebut digunakan untuk melatih data dan melakukan prediksi terhadap data uji.

```
best_knn = grid.best_estimator_  
best_knn.fit(x_train_smote, y_train_smote)  
  
# Prediksi  
y_pred = best_knn.predict(x_test_scaled)
```

Hasil prediksi (`y_pred`) nantinya akan dibandingkan dengan label asli (`y_test`) untuk mengevaluasi performa model menggunakan confusion matrix dan classification report di tahap berikutnya.

10. Evaluasi Model dengan Classification Report

Setelah model KNN melakukan prediksi terhadap data uji, performanya dievaluasi menggunakan metrik-metrik utama dari `classification_report`.

```
print("\nClassification Report:")  
print(classification_report(y_test, y_pred))
```

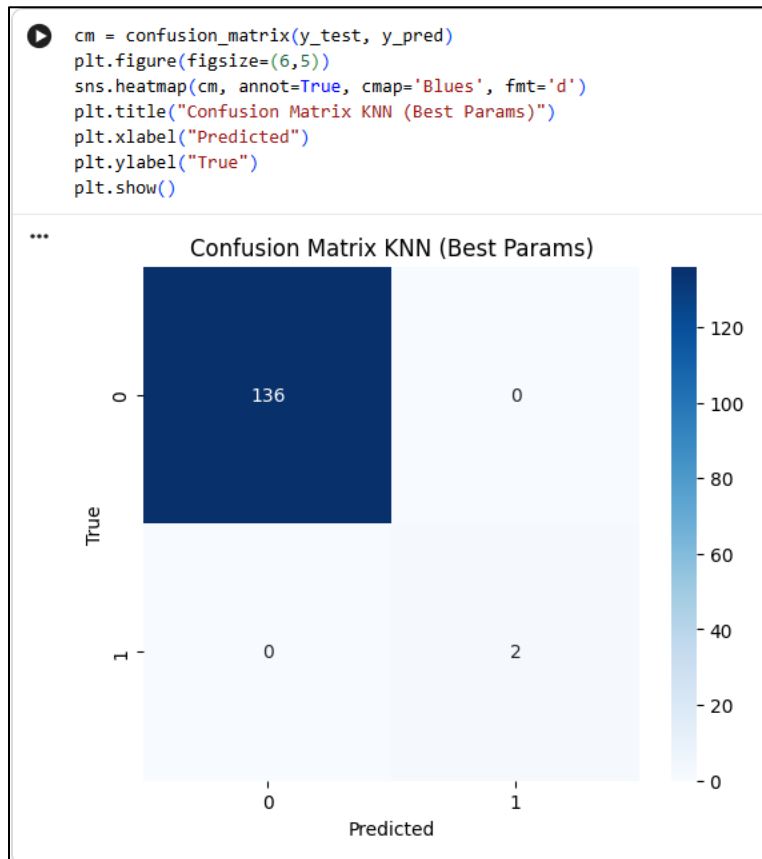
```
Classification Report:  
              precision    recall  f1-score   support  
  
      0               1.00      1.00      1.00       136  
      1               1.00      1.00      1.00         2  
  
   accuracy               1.00       138  
  macro avg               1.00       138  
 weighted avg               1.00       138
```

Output Kode;

- Model memiliki **akurasi sempurna (100%)** untuk kedua kelas, baik *Tepat Waktu (0)* maupun *Terlambat (1)*.
- Semua metrik — **precision**, **recall**, dan **f1-score** — bernilai **1.00**, menandakan model mampu mengenali kedua kelas dengan sangat baik.
- Hal ini menunjukkan bahwa kombinasi **data seimbang (SMOTE)** dan **parameter optimal KNN** menghasilkan performa yang sangat tinggi pada data uji.

11. Evaluasi dengan Confusion Matrix

Confusion Matrix digunakan untuk menunjukkan seberapa baik model memprediksi setiap kelas (tepat waktu vs terlambat).

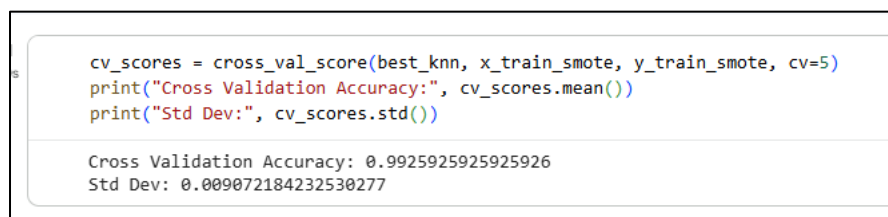


Output Kode;

- 136 mahasiswa “Tepat Waktu” (0) diprediksi dengan benar.
- 2 mahasiswa “Terlambat” (1) juga diprediksi dengan benar.
- Tidak ada kesalahan klasifikasi (nilai 0 di luar diagonal).

12. Validasi Model dengan Cross Validation

Setelah model KNN terbaik dilatih dan diuji, langkah selanjutnya adalah melakukan Cross Validation (CV) untuk memastikan model tidak overfitting dan performanya konsisten di berbagai subset data.



Output Kode;

- Rata-rata akurasi model dari 5 percobaan cross-validation adalah **99.29%**.

- Nilai standar deviasi yang sangat kecil (**0.00097**) menunjukkan hasil model **stabil dan konsisten** di setiap subset data.

Dengan hasil ini, model KNN yang digunakan **sangat andal**, tidak mengalami overfitting, dan performanya tinggi pada data yang berbeda.

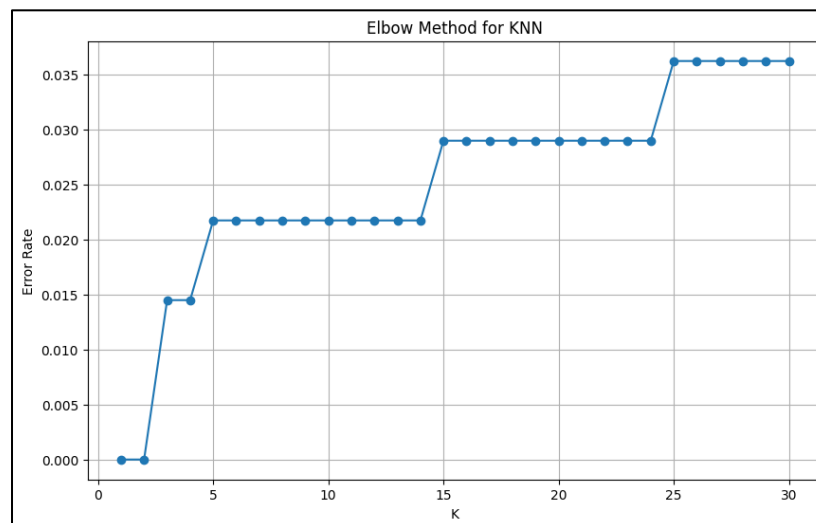
13. Menentukan Nilai K Optimal dengan Elbow Method

Elbow Method digunakan untuk membantu menentukan jumlah tetangga (K) yang paling optimal bagi algoritma K-Nearest Neighbors (KNN). Prinsipnya adalah mencari titik di mana penurunan tingkat error mulai melambat — titik tersebut dianggap sebagai “siku” (elbow).

```
errors = []

for k in range(1, 31):
    knn_temp = KNeighborsClassifier(n_neighbors=k)
    knn_temp.fit(x_train_smote, y_train_smote)
    pred = knn_temp.predict(x_test_scaled)
    errors.append(np.mean(pred != y_test))

plt.figure(figsize=(10,6))
plt.plot(range(1,31), errors, marker='o')
plt.title("Elbow Method for KNN")
plt.xlabel("K")
plt.ylabel("Error Rate")
plt.grid(True)
plt.show()
```



Grafik menunjukkan bahwa:

- Error rate paling rendah terjadi pada **K kecil (sekitar 1–3)**.
- Setelah itu, error rate cenderung stabil dan sedikit meningkat.

Dengan demikian, **K = 1** merupakan pilihan paling optimal — sejalan dengan hasil **GridSearchCV** sebelumnya. Model dengan **K kecil** mampu menangkap pola data dengan sangat baik setelah dataset diseimbangkan menggunakan **SMOTE**.

Tugas Praktikum Mandiri

Tugas praktikum mandiri tersedia di bagian akhir slide presentasi Mata Kuliah Machine Learning materi Algoritma K-Nearest Neighbors